Build a registrar workflow state machine and RBAC permission matrix for department drafts, Academic Affairs sign-off, and final publishing

Computer
Build an immutable audit logging and validation hook system for FERPA-compliant academic scheduling approvals and registrar state changes
Computer
Show the complete JSON state machine schema
Write the TypeScript state transition engine
Map out the notification and webhook system

Below is a complete reference design for Semester's **registrar workflow state machine**, **RBAC matrix**, **immutable audit hooks**, **JSON workflow schema**, **TypeScript transition engine**, and **notification/webhook architecture.** Academic scheduling workflows are commonly used to route department changes and exceptions to the right approvers, while FERPA-oriented systems require granular access control and comprehensive audit trails for record access and disclosure-related activity. [1] [2] [3]

JSON state machine schema

Use an explicit machine definition so workflow policy is data-driven rather than buried in code. Workflow systems in academic scheduling commonly define steps, participants, and conditional routing rules, which maps well to a declarative machine model. [1]

```
{
  "machine": "registrar_schedule_workflow",
  "version": 1,
  "initialState": "draft",
  "terminalStates": ["published", "locked", "denied", "archived"],
  "roles": [
    "department_scheduler",
    "department_chair",
    "dean",
    "academic_affairs",
    "registrar_scheduler",
    "registrar_admin",
    "room_manager",
    "auditor",
    "system"
  ],
  "states": {
    "draft": {
```

```
      "label": "Draft",
      "editable": true,
      "allowedActions": ["save", "submit", "archive"],
      "transitions": [
        {
          "action": "submit",
          "to": "validation_pending",
          "actors": ["department_scheduler", "registrar_scheduler", "registrar_admin"
          "guards": ["requiredFieldsPresent"],
          "hooks": ["auditTransition", "queueValidation"]
        },
        {
          "action": "archive",
          "to": "archived",
          "actors": ["registrar_admin"],
          "hooks": ["auditTransition"]
        }
      ]
    },
    "validation_pending": {
      "label": "Validation Pending",
      "editable": false,
      "allowedActions": ["validation_pass", "validation_fail"],
      "transitions": [
        {
          "action": "validation_pass",
          "to": "department_ready",
          "actors": ["system"],
          "guards": ["noBlockingConflicts"],
          "hooks": ["auditTransition", "createWorkflowTask"]
        },
        {
          "action": "validation_fail",
          "to": "validation_failed",
          "actors": ["system"],
          "hooks": ["auditTransition", "notifySubmitter"]
        }
      ]
    },
    "validation_failed": {
      "label": "Validation Failed",
      "editable": true,
      "allowedActions": ["resubmit", "request_exception"],
      "transitions": [
        {
          "action": "resubmit",
          "to": "validation_pending",
          "actors": ["department_scheduler", "registrar_scheduler", "registrar_admin"
          "guards": ["requiredFieldsPresent"],
          "hooks": ["auditTransition", "queueValidation"]
        },
        {
          "action": "request_exception",
          "to": "academic_affairs_review",
          "actors": ["department_scheduler", "department_chair", "dean", "registrar_s
          "guards": ["exceptionEligible"],
```

```
        "hooks": ["auditTransition", "createWorkflowTask"]
      }
    ]
  },
  "department_ready": {
    "label": "Department Ready",
    "editable": false,
    "allowedActions": ["start_chair_review"],
    "transitions": [
      {
        "action": "start_chair_review",
        "to": "chair_review",
        "actors": ["system", "department_scheduler", "registrar_scheduler"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      }
    ]
  },
  "chair_review": {
    "label": "Chair Review",
    "editable": false,
    "allowedActions": ["approve", "return", "route_exception"],
    "transitions": [
      {
        "action": "approve",
        "to": "dean_review",
        "actors": ["department_chair"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      },
      {
        "action": "return",
        "to": "draft",
        "actors": ["department_chair"],
        "guards": ["commentRequired"],
        "hooks": ["auditTransition", "notifySubmitter"]
      },
      {
        "action": "route_exception",
        "to": "academic_affairs_review",
        "actors": ["department_chair"],
        "guards": ["exceptionEligible"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      }
    ]
  },
  "dean_review": {
    "label": "Dean Review",
    "editable": false,
    "allowedActions": ["approve", "return", "route_exception"],
    "transitions": [
      {
        "action": "approve",
        "to": "registrar_review",
        "actors": ["dean"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      },
      {
```

```
        "action": "return",
        "to": "draft",
        "actors": ["dean"],
        "guards": ["commentRequired"],
        "hooks": ["auditTransition", "notifySubmitter"]
      },
      {
        "action": "route_exception",
        "to": "academic_affairs_review",
        "actors": ["dean"],
        "guards": ["exceptionEligible"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      }
    ]
  },
  "registrar_review": {
    "label": "Registrar Review",
    "editable": false,
    "allowedActions": ["approve", "return", "route_exception"],
    "transitions": [
      {
        "action": "approve",
        "to": "room_coordination",
        "actors": ["registrar_scheduler", "registrar_admin"],
        "guards": ["needsCentralRoom"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      },
      {
        "action": "approve",
        "to": "approved_for_publish",
        "actors": ["registrar_scheduler", "registrar_admin"],
        "guards": ["publishableWithoutRoomStep"],
        "hooks": ["auditTransition"]
      },
      {
        "action": "return",
        "to": "draft",
        "actors": ["registrar_scheduler", "registrar_admin"],
        "guards": ["commentRequired"],
        "hooks": ["auditTransition", "notifySubmitter"]
      },
      {
        "action": "route_exception",
        "to": "academic_affairs_review",
        "actors": ["registrar_scheduler", "registrar_admin"],
        "guards": ["exceptionEligible"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      }
    ]
  },
  "room_coordination": {
    "label": "Room Coordination",
    "editable": false,
    "allowedActions": ["assign_and_clear", "return"],
    "transitions": [
      {
```

```
          "action": "assign_and_clear",
          "to": "approved_for_publish",
          "actors": ["room_manager", "registrar_scheduler", "registrar_admin"],
          "guards": ["roomResolved"],
          "hooks": ["auditTransition"]
        },
        {
          "action": "return",
          "to": "registrar_review",
          "actors": ["room_manager", "registrar_scheduler", "registrar_admin"],
          "guards": ["commentRequired"],
          "hooks": ["auditTransition"]
        }
      ]
    },
    "academic_affairs_review": {
      "label": "Academic Affairs Review",
      "editable": false,
      "allowedActions": ["approve", "deny", "return"],
      "transitions": [
        {
          "action": "approve",
          "to": "registrar_review",
          "actors": ["academic_affairs"],
          "hooks": ["auditTransition", "createWorkflowTask"]
        },
        {
          "action": "deny",
          "to": "denied",
          "actors": ["academic_affairs"],
          "guards": ["commentRequired"],
          "hooks": ["auditTransition", "notifySubmitter"]
        },
        {
          "action": "return",
          "to": "draft",
          "actors": ["academic_affairs"],
          "guards": ["commentRequired"],
          "hooks": ["auditTransition", "notifySubmitter"]
        }
      ]
    },
    "approved_for_publish": {
      "label": "Approved for Publish",
      "editable": false,
      "allowedActions": ["publish", "return"],
      "transitions": [
        {
          "action": "publish",
          "to": "published",
          "actors": ["registrar_admin"],
          "guards": ["prepublishChecksPassed"],
          "hooks": ["snapshotPublication", "auditTransition", "emitPublicationEvent"
        },
        {
          "action": "return",
```

```
        "to": "draft",
        "actors": ["registrar_admin"],
        "guards": ["commentRequired"],
        "hooks": ["auditTransition", "notifySubmitter"]
      }
    ]
  },
  "published": {
    "label": "Published",
    "editable": false,
    "allowedActions": ["request_change", "lock"],
    "transitions": [
      {
        "action": "request_change",
        "to": "change_requested",
        "actors": ["department_scheduler", "department_chair", "dean", "registrar_s
        "hooks": ["auditTransition", "createWorkflowTask"]
      },
      {
        "action": "lock",
        "to": "locked",
        "actors": ["registrar_admin"],
        "hooks": ["auditTransition"]
      }
    ]
  },
  "change_requested": {
    "label": "Change Requested",
    "editable": false,
    "allowedActions": ["reopen_validation", "reopen_registrar_review"],
    "transitions": [
      {
        "action": "reopen_validation",
        "to": "validation_pending",
        "actors": ["registrar_scheduler", "registrar_admin"],
        "hooks": ["auditTransition", "queueValidation"]
      },
      {
        "action": "reopen_registrar_review",
        "to": "registrar_review",
        "actors": ["registrar_scheduler", "registrar_admin"],
        "hooks": ["auditTransition", "createWorkflowTask"]
      }
    ]
  },
  "locked": {
    "label": "Locked",
    "editable": false,
    "allowedActions": ["emergency_unlock", "archive"],
    "transitions": [
      {
        "action": "emergency_unlock",
        "to": "registrar_review",
        "actors": ["registrar_admin"],
        "guards": ["commentRequired"],
        "hooks": ["auditTransition", "createWorkflowTask"]
```

```
        },
        {
          "action": "archive",
          "to": "archived",
          "actors": ["registrar_admin", "system"],
          "hooks": ["auditTransition"]
        }
      ]
    },
    "denied": {
      "label": "Denied",
      "editable": false,
      "allowedActions": ["rework", "archive"],
      "transitions": [
        {
          "action": "rework",
          "to": "draft",
          "actors": ["department_scheduler", "registrar_admin"],
          "hooks": ["auditTransition"]
        },
        {
          "action": "archive",
          "to": "archived",
          "actors": ["registrar_admin", "system"],
          "hooks": ["auditTransition"]
        }
      ]
    },
    "archived": {
      "label": "Archived",
      "editable": false,
      "allowedActions": [],
      "transitions": []
    }
  }
}
```

TypeScript transition engine

This engine should validate actor role, current state, guards, and hooks, then persist the state change and immutable audit record in one transaction. FERPA-oriented systems rely on audit logs and granular access controls, so transition execution should always produce auditable evidence.[3] [4]

```
export type WorkflowState =
  | "draft"
  | "validation_pending"
  | "validation_failed"
  | "department_ready"
  | "chair_review"
  | "dean_review"
  | "registrar_review"
  | "room_coordination"
  | "academic_affairs_review"
```

```
  | "approved_for_publish"
  | "published"
  | "change_requested"
  | "locked"
  | "denied"
  | "archived";

export type WorkflowRole =
  | "department_scheduler"
  | "department_chair"
  | "dean"
  | "academic_affairs"
  | "registrar_scheduler"
  | "registrar_admin"
  | "room_manager"
  | "auditor"
  | "system";

export type WorkflowAction =
  | "save"
  | "submit"
  | "archive"
  | "validation_pass"
  | "validation_fail"
  | "resubmit"
  | "request_exception"
  | "start_chair_review"
  | "approve"
  | "return"
  | "route_exception"
  | "assign_and_clear"
  | "deny"
  | "publish"
  | "request_change"
  | "lock"
  | "reopen_validation"
  | "reopen_registrar_review"
  | "emergency_unlock"
  | "rework";

export interface TransitionContext {
  entityId: string;
  workflowInstanceId: string;
  state: WorkflowState;
  actorUserId: string;
  actorRole: WorkflowRole;
  scopeType: string;
  scopeId: string;
  comment?: string;
  requestId: string;
  metadata?: Record<string, unknown>;
}

export interface TransitionDefinition {
  action: WorkflowAction;
  to: WorkflowState;
```

```
  actors: WorkflowRole[];
  guards?: string[];
  hooks?: string[];
}

export interface StateDefinition {
  label: string;
  editable: boolean;
  allowedActions: WorkflowAction[];
  transitions: TransitionDefinition[];
}

export interface MachineDefinition {
  machine: string;
  version: number;
  initialState: WorkflowState;
  terminalStates: WorkflowState[];
  states: Record<WorkflowState, StateDefinition>;
}

export interface GuardServices {
  requiredFieldsPresent(ctx: TransitionContext): Promise<boolean>;
  noBlockingConflicts(ctx: TransitionContext): Promise<boolean>;
  exceptionEligible(ctx: TransitionContext): Promise<boolean>;
  commentRequired(ctx: TransitionContext): Promise<boolean>;
  needsCentralRoom(ctx: TransitionContext): Promise<boolean>;
  publishableWithoutRoomStep(ctx: TransitionContext): Promise<boolean>;
  roomResolved(ctx: TransitionContext): Promise<boolean>;
  prepublishChecksPassed(ctx: TransitionContext): Promise<boolean>;
}

export interface HookServices {
  auditTransition(input: AuditInput): Promise<void>;
  queueValidation(ctx: TransitionContext): Promise<void>;
  createWorkflowTask(ctx: TransitionContext, nextState: WorkflowState): Promise<void>
  notifySubmitter(ctx: TransitionContext, nextState: WorkflowState): Promise<void>;
  snapshotPublication(ctx: TransitionContext): Promise<void>;
  emitPublicationEvent(ctx: TransitionContext): Promise<void>;
}

export interface AuditInput {
  actorUserId: string;
  actedAsRoleCode: WorkflowRole;
  action: WorkflowAction;
  fromState: WorkflowState;
  toState: WorkflowState;
  workflowInstanceId: string;
  entityId: string;
  scopeType: string;
  scopeId: string;
  comment?: string;
  metadata?: Record<string, unknown>;
  requestId: string;
}

export interface WorkflowRepository {
```

```
  getMachine(): Promise<MachineDefinition>;
  updateState(
    workflowInstanceId: string,
    fromState: WorkflowState,
    toState: WorkflowState
  ): Promise<boolean>;
  appendTransition(input: AuditInput): Promise<void>;
  inTransaction<T>(fn: () => Promise<T>): Promise<T>;
}

export class TransitionError extends Error {
  constructor(
    message: string,
    public code: string
  ) {
    super(message);
  }
}

export class WorkflowEngine {
  constructor(
    private readonly repo: WorkflowRepository,
    private readonly guards: GuardServices,
    private readonly hooks: HookServices
  ) {}

  async transition(
    ctx: TransitionContext,
    action: WorkflowAction
  ): Promise<WorkflowState> {
    const machine = await this.repo.getMachine();
    const stateDef = machine.states[ctx.state];

    if (!stateDef) {
      throw new TransitionError("Unknown state", "STATE_UNKNOWN");
    }

    const candidates = stateDef.transitions.filter(
      t => t.action === action && t.actors.includes(ctx.actorRole)
    );

    if (candidates.length === 0) {
      throw new TransitionError(
        "Action not allowed for role/state",
        "ACTION_NOT_ALLOWED"
      );
    }

    let selected: TransitionDefinition | null = null;

    for (const candidate of candidates) {
      const ok = await this.evaluateGuards(
        ctx,
        candidate.guards ?? []
      );
      if (ok) {
```

```
        selected = candidate;
        break;
      }
    }

    if (!selected) {
      throw new TransitionError(
        "No transition guards passed",
        "GUARD_FAILED"
      );
    }

    const toState = selected.to;

    await this.repo.inTransaction(async () => {
      const moved = await this.repo.updateState(
        ctx.workflowInstanceId,
        ctx.state,
        toState
      );

      if (!moved) {
        throw new TransitionError(
          "State changed concurrently",
          "CONCURRENT_MODIFICATION"
        );
      }

      const audit: AuditInput = {
        actorUserId: ctx.actorUserId,
        actedAsRoleCode: ctx.actorRole,
        action,
        fromState: ctx.state,
        toState,
        workflowInstanceId: ctx.workflowInstanceId,
        entityId: ctx.entityId,
        scopeType: ctx.scopeType,
        scopeId: ctx.scopeId,
        comment: ctx.comment,
        metadata: ctx.metadata,
        requestId: ctx.requestId
      };

      await this.repo.appendTransition(audit);

      if (selected.hooks?.includes("auditTransition")) {
        await this.hooks.auditTransition(audit);
      }
      if (selected.hooks?.includes("snapshotPublication")) {
        await this.hooks.snapshotPublication(ctx);
      }
      if (selected.hooks?.includes("queueValidation")) {
        await this.hooks.queueValidation(ctx);
      }
      if (selected.hooks?.includes("createWorkflowTask")) {
        await this.hooks.createWorkflowTask(ctx, toState);
```

```
      }
      if (selected.hooks?.includes("notifySubmitter")) {
        await this.hooks.notifySubmitter(ctx, toState);
      }
      if (selected.hooks?.includes("emitPublicationEvent")) {
        await this.hooks.emitPublicationEvent(ctx);
      }
    });

    return toState;
  }

  private async evaluateGuards(
    ctx: TransitionContext,
    guardNames: string[]
  ): Promise<boolean> {
    for (const name of guardNames) {
      const fn = this.guards[name as keyof GuardServices];
      if (!fn) {
        throw new TransitionError(
          `Unknown guard: ${name}`,
          "GUARD_UNKNOWN"
        );
      }
      const ok = await fn.call(this.guards, ctx);
      if (!ok) return false;
    }
    return true;
  }
}
```

Implementation notes

- Persist the workflow state change and transition append in one transaction.
- Use optimistic concurrency on `updateState()` so stale UIs cannot overwrite newer state.
- Treat `appendTransition()` as immutable, never an update-in-place.
- Run expensive notifications or webhooks asynchronously after commit, but record the transition synchronously.

Database schema

The schema should keep academic data, workflow, RBAC, validation, audit, and event delivery separate. FERPA-oriented compliance guidance emphasizes detailed audit logging, access control, and controlled disclosure records.[5] [3]

Workflow and RBAC

```
CREATE TABLE roles (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL
```

```
);

CREATE TABLE permissions (
  id uuid PRIMARY KEY,
  resource text NOT NULL,
  action text NOT NULL,
  UNIQUE(resource, action)
);

CREATE TABLE role_permissions (
  role_id uuid NOT NULL REFERENCES roles(id),
  permission_id uuid NOT NULL REFERENCES permissions(id),
  PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE role_bindings (
  id uuid PRIMARY KEY,
  user_id uuid NOT NULL,
  role_id uuid NOT NULL REFERENCES roles(id),
  scope_type text NOT NULL CHECK (
    scope_type IN ('institution','campus','college','department','program','room_pool
  ),
  scope_id uuid NOT NULL,
  starts_at timestamptz,
  ends_at timestamptz,
  granted_by uuid NOT NULL,
  justification text,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE workflow_instances (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section','term_calendar_event'))
  entity_id uuid NOT NULL,
  current_state text NOT NULL,
  version integer NOT NULL DEFAULT 1,
  initiated_by uuid NOT NULL,
  started_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE workflow_tasks (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  assigned_role_code text NOT NULL,
  assigned_scope_type text NOT NULL,
  assigned_scope_id uuid,
  assigned_user_id uuid,
  task_type text NOT NULL,
  status text NOT NULL CHECK (status IN ('open','approved','returned','denied','cance
  due_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  resolved_at timestamptz
);

CREATE TABLE workflow_transitions (
```

```
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  entity_id uuid NOT NULL,
  from_state text NOT NULL,
  to_state text NOT NULL,
  action text NOT NULL,
  acted_by uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  scope_type text NOT NULL,
  scope_id uuid NOT NULL,
  justification text,
  request_id text NOT NULL,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Validation and conflict tracking

```
CREATE TABLE validation_runs (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section','term')),
  entity_id uuid NOT NULL,
  trigger_type text NOT NULL CHECK (
    trigger_type IN ('save','submit','transition','prepublish','batch')
  ),
  status text NOT NULL CHECK (status IN ('passed','failed','warning_only')),
  blocking_count integer NOT NULL DEFAULT 0,
  warning_count integer NOT NULL DEFAULT 0,
  created_by uuid,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE scheduling_conflicts (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL,
  conflict_type text NOT NULL CHECK (
    conflict_type IN (
      'instructor_overlap','room_overlap','capacity_mismatch',
      'pattern_violation','contact_hour_mismatch','feature_mismatch',
      'cross_list_mismatch'
    )
  ),
  severity text NOT NULL CHECK (severity IN ('warning','blocking')),
  left_section_id uuid NOT NULL,
  right_section_id uuid,
  room_id uuid,
  instructor_id uuid,
  status text NOT NULL CHECK (status IN ('open','waived','resolved')),
  opened_by_run_id uuid REFERENCES validation_runs(id),
  waiver_reason text,
  waived_by uuid,
  resolved_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Immutable audit and FERPA tracking

```
CREATE TABLE audit_events (
  id uuid PRIMARY KEY,
  actor_user_id uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  module text NOT NULL,
  action text NOT NULL,
  target_type text NOT NULL,
  target_id uuid NOT NULL,
  scope_type text NOT NULL,
  scope_id uuid NOT NULL,
  legitimate_interest text,
  reason_code text,
  request_id text NOT NULL,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  ip_address inet,
  user_agent text,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ferpa_access_log (
  id uuid PRIMARY KEY,
  actor_user_id uuid NOT NULL,
  record_type text NOT NULL,
  record_id uuid NOT NULL,
  access_type text NOT NULL CHECK (
    access_type IN ('view','edit','export','approve','disclose','admin_override')
  ),
  legitimate_interest text NOT NULL,
  request_id text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ferpa_disclosure_requests (
  id uuid PRIMARY KEY,
  student_id uuid NOT NULL,
  requester_identity text NOT NULL,
  requested_record_type text NOT NULL,
  purpose text NOT NULL,
  status text NOT NULL CHECK (
    status IN ('submitted','under_review','approved','denied','fulfilled')
  ),
  reviewed_by uuid,
  decision_reason text,
  created_at timestamptz NOT NULL DEFAULT now(),
  decided_at timestamptz,
  fulfilled_at timestamptz
);
```

Immutability controls

Use append-only tables for `workflow_transitions`, `audit_events`, and `ferpa_access_log`. Compliance guidance emphasizes durable audit logs for student-record access and disclosure activity.[4] [6]

Recommended database controls:

- Revoke `UPDATE` and `DELETE` on audit tables for app roles.
- Write through stored procedures or a dedicated service account.
- Add hash chaining or periodic digesting for tamper evidence.
- Replicate audit tables to separate immutable storage.

API endpoints for transitions and validation

Workflow APIs should use command-style endpoints so every state change is explicit, validated, and audited. Webhook and event-driven guidance also recommends clear event identity and explicit processing states for reliable workflow systems.[7] [8]

Transition endpoints

```
POST /api/sections/{sectionId}/submit
POST /api/sections/{sectionId}/resubmit
POST /api/sections/{sectionId}/request-exception
POST /api/workflow/tasks/{taskId}/approve
POST /api/workflow/tasks/{taskId}/return
POST /api/workflow/tasks/{taskId}/deny
POST /api/workflow/tasks/{taskId}/route-academic-affairs
POST /api/sections/{sectionId}/request-change
POST /api/sections/{sectionId}/lock
POST /api/sections/{sectionId}/unlock-emergency
POST /api/terms/{termId}/publish
POST /api/terms/{termId}/archive
```

Validation endpoints

```
POST /api/sections/{sectionId}/validate
GET  /api/sections/{sectionId}/conflicts
GET  /api/conflicts?termId=&scopeId=&severity=&status=
POST /api/conflicts/{conflictId}/waive
POST /api/conflicts/{conflictId}/resolve
GET  /api/rooms/availability?termId=&day=&start=&end=&capacity=
POST /api/terms/{termId}/prepublish-check
GET  /api/terms/{termId}/publication-preview
```

Audit and FERPA endpoints

```
GET  /api/audit-events?targetType=section&targetId=
GET  /api/audit-events?actorId=&module=registrar
GET  /api/ferpa/access-log?recordType=&recordId=
POST /api/ferpa/disclosure-requests
POST /api/ferpa/disclosure-requests/{id}/decide
POST /api/exports/sections
```

Notification and webhook system

Event-driven architecture is a strong fit here because workflow transitions naturally produce business events that should trigger notifications, integrations, and downstream automation without coupling every system directly. Webhook and EDA guidance emphasizes asynchronous processing, endpoint registration, retry logic, event identity, authentication, and duplicate handling. [9] [10] [7]

Event flow

```
User or system action
  → Workflow transition committed
  → Transition + audit log appended
  → Outbox event written in same transaction
  → Event dispatcher publishes event
  → Notification service + webhook service consume event
  → Delivery attempts tracked
  → Retry / suppress / quarantine on failure
```

Core event types

Event type	Trigger
`section.submitted`	Department submits a section
`section.validation_failed`	Validation returns blocking issues
`workflow.task_assigned`	New approver task created
`workflow.task_approved`	Approver completes a step
`workflow.task_returned`	Section returned for rework
`section.exception_routed`	Sent to Academic Affairs
`section.room_assigned`	Room coordination completed
`term.schedule_published`	Official publish finished
`records.export_requested`	Sensitive export initiated
`ferpa.disclosure_decided`	Disclosure request approved or denied

Notification architecture

Component	Responsibility
Outbox table	Stores domain events atomically with workflow commit
Event dispatcher	Publishes queued events to subscribers
Notification policy engine	Determines recipients, channels, suppression, escalation
Template renderer	Renders in-app, email, and SMS messages
Delivery workers	Send and retry channel deliveries
Webhook publisher	Sends signed outbound events to subscribed systems
Delivery ledger	Tracks attempts, failures, suppressions, and reads

Example notification policies

- `workflow.task_assigned`: in-app immediately, email within 1 minute.
- `workflow.task_returned`: in-app + email to submitter.
- `section.validation_failed`: in-app only by default, unless SLA breach.
- `term.schedule_published`: email to department schedulers and webhook to downstream SIS/LMS integrations.
- `records.export_requested`: notify compliance reviewer and requester.
- `ferpa.disclosure_decided`: notify privacy/compliance assignee only.

Webhook model

```
{
  "id": "evt_01J...",
  "type": "term.schedule_published",
  "occurredAt": "2026-10-06T20:11:00Z",
  "source": "semester.registrar",
  "schemaVersion": 1,
  "aggregate": {
    "type": "term",
    "id": "term_123"
  },
  "data": {
    "termId": "term_123",
    "publishVersion": 7,
    "publishedBy": "user_456"
  }
}
```

Webhook delivery requirements

- Subscriber endpoint registration.
- Per-subscriber secret or signing key.
- HMAC signature header.
- At-least-once delivery.
- Idempotency key on every event.
- Retry with backoff for transient failures.
- Dead-letter/quarantine state for repeated failure.
- Delivery attempt history with timestamps and HTTP outcomes.

Webhook architecture guidance recommends explicit event identity, validation, retries, authentication, and observable recovery paths.[8] [7]

Delivery schema

```
CREATE TABLE outbox_events (
  id uuid PRIMARY KEY,
  event_type text NOT NULL,
  aggregate_type text NOT NULL,
  aggregate_id uuid NOT NULL,
  payload jsonb NOT NULL,
  status text NOT NULL CHECK (status IN ('pending','published','failed')),
  created_at timestamptz NOT NULL DEFAULT now(),
  published_at timestamptz
);

CREATE TABLE webhook_subscriptions (
  id uuid PRIMARY KEY,
  subscriber_name text NOT NULL,
  callback_url text NOT NULL,
  signing_secret text NOT NULL,
  enabled boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE webhook_deliveries (
  id uuid PRIMARY KEY,
  outbox_event_id uuid NOT NULL REFERENCES outbox_events(id),
  webhook_subscription_id uuid NOT NULL REFERENCES webhook_subscriptions(id),
  status text NOT NULL CHECK (
    status IN ('queued','sent','failed','quarantined','duplicate')
  ),
  attempt_count integer NOT NULL DEFAULT 0,
  last_http_status integer,
  next_attempt_at timestamptz,
  last_error text,
  created_at timestamptz NOT NULL DEFAULT now(),
  delivered_at timestamptz
);
```

Implementation guidance

The safest implementation pattern is:

1. **Transition engine** validates state + role + guards.
2. **Database transaction** updates current state and appends immutable transition/audit rows.
3. **Outbox event** is written in the same transaction.
4. **Async processors** deliver notifications and webhooks.
5. **FERPA access log** records all sensitive view/export/override actions separately from workflow events.

That gives Semester a registrar-grade workflow backbone with explicit approval routing, immutable evidence, and scalable event delivery. [7] [4] [1]

⁂

1. https://coursedog.my.site.com/support/s/article/WORKFLOWSWorkflowBasics67fd8e827574b
2. https://it.osu.edu/student-information-system-sis/sis-room-class-scheduling/1-course-class-workflow
3. https://openeducat.org/features/ferpa-compliance/
4. https://secureprivacy.ai/blog/ferpa-compliance-software
5. https://www.codetrade.io/blog/ferpa-compliant-lms-us-universities/
6. https://edves.us/compliance/ferpa-and-student-data-privacy/
7. https://datamam.com/webhook-architecture-event-driven-api-integrations/
8. https://dev.to/vikthurrdev/designing-a-webhook-service-a-practical-guide-to-event-driven-architecture-3lep
9. https://apidog.com/blog/comprehensive-guide-to-webhooks-and-eda/
10. https://softwarepatternslexicon.com/js/integration-with-other-systems/webhooks-and-event-driven-integration/
11. https://registrar.vanderbilt.edu/faculty-staff/academic-scheduling.php
12. https://agentplace.io/blog/webhook-and-event-driven-agent-architecture-real-time-automation
13. https://help.openai.com/en/articles/10291617-tasks-in-chatgpt
14. https://code.zeba.academy/building-event-driven-api/